Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No: 92301733021**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly
actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!).
There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	**stemmed words**
0	connect	connect
'I	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No: 92301733021**

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

To be attached with

```
# NLP EXPERIMENT FULL CODE (FINAL FIXED)
import nltk
import string
import spacy
# DOWNLOAD REQUIRED NLTK DATA
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, RegexpParser

# LOAD SPACY MODEL
nlp = spacy.load("en_core_web_sm")

# INPUT TEXT
text = input("Enter a sentence: ")
print("\nOriginal Text:\n", text)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No: 92301733021**

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
Enter a sentence: Dolls powered by artificial intelligence that can just up and get a mind of their own one day.

Original Text:
 Dolls powered by artificial intelligence that can just up and get a mind of their own one day.
```

1. Tokenization

Code:

```
tokens = word_tokenize(text)
print("\n1. Tokens:\n", tokens)
```

Output:

```
1. Tokens:
 ['Dolls', 'powered', 'by', 'artificial', 'intelligence', 'that', 'can', 'just', 'up', 'and', 'get', 'a', 'mind', 'of', 'their', 'own', 'one', 'day', '.']
```

2. Filteration

Code:

```
filtered_tokens = [word for word in tokens if word not in string.punctuation]
print("\n2. Filtered Tokens:\n", filtered_tokens)
```

Output:

```
2. Filtered Tokens:
 ['Dolls', 'powered', 'by', 'artificial', 'intelligence', 'that', 'can', 'just', 'up', 'and', 'get', 'a', 'mind', 'of', 'their', 'own', 'one', 'day']
```

3. StopWords Removal

Code:

```
stop_words = set(stopwords.words('english'))
clean_tokens = [word for word in filtered_tokens if word.lower() not in stop_words]
print("\n3. After Stopword Removal:\n", clean_tokens)
```

Output:

```
3. After Stopword Removal:
 ['Dolls', 'powered', 'artificial', 'intelligence', 'get', 'mind', 'one', 'day']
```

4. PoS Tagging

Code:

```
pos_tags = pos_tag(clean_tokens)
print("\n4. POS Tags:\n", pos_tags)
```

Output:

```
4. POS Tags:
 [('Dolls', 'NNS'), ('powered', 'VBD'), ('artificial', 'JJ'), ('intelligence', 'NN'), ('get', 'NN'), ('mind', 'VB'), ('one', 'CD'), ('day', 'NN')]
```

5. Noun Phrase Chunking

Code:

```
grammar = "NP: {<DT>?<JJ>*<NN.*>+}"
chunk_parser = RegexpParser(grammar)
chunk_tree = chunk_parser.parse(pos_tags)
print("\n5. Noun Phrase Chunking:\n")
print(chunk_tree)
```

Output:

```
5. Noun Phrase Chunking:

(S
  (NP Dolls/NNS)
  powered/VBD
  (NP artificial/JJ intelligence/NN get/NN)
  mind/VB
  one/CD
  (NP day/NN))
```

6. Dependancy Parsing

Code:

```
doc = nlp(text)
print("\n6. Dependency Parsing:\n")
for token in doc:
    print(f"{token.text} --> {token.dep_} --> {token.head.text}")
```

Output:

```
6. Dependency Parsing:

Dolls --> ROOT --> Dolls
powered --> acl --> Dolls
by --> agent --> powered
artificial --> amod --> intelligence
intelligence --> pobj --> by
that --> nsubj --> up
can --> aux --> up
just --> advmod --> up
up --> relcl --> Dolls
and --> cc --> up
get --> conj --> up
a --> det --> mind
mind --> dobj --> get
of --> prep --> mind
their --> poss --> day
own --> amod --> day
one --> nummod --> day
day --> pobj --> of
. --> punct --> Dolls
```

Observation:

The input sentence was processed using NLTK and spaCy. Tokenization split the text into words, filtration and stopword removal cleaned the data, POS tagging identified grammatical roles, noun phrase chunking grouped meaningful phrases, and dependency parsing showed relationships between words. The experiment successfully demonstrated effective text preprocessing and analysis.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No: 92301733021**

Post Lab Exercise:

1. **Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.**
   Code:

```
# Take your own document of 10 sentences and perform the same tasks. Attach code
    and screenshot of your output.
import nltk
import string
import spacy

# Download required data
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, RegexpParser
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Load spacy
nlp = spacy.load("en_core_web_sm")

# INPUT DOCUMENT (10 sentences)
text = """Peacock is beautiful bird
Sparrow is a small bird
Dog is a loyal animal
Cat drinks milk daily
```

```
Sun gives us light
Moon shines at night
Boy plays in garden
Girl reads a book
Water is very important
Trees give us oxygen"""

print("Original Text:\n", text)

# TOKENIZATION
tokens = word_tokenize(text)
print("\nTokens:\n", tokens)

# FILTERING
filtered = [w for w in tokens if w not in string.punctuation]

# STOPWORDS REMOVAL
stop_words = set(stopwords.words('english'))
clean_tokens = [w for w in filtered if w.lower() not in stop_words]
print("\nClean Tokens:\n", clean_tokens)

# POS TAGGING
pos_tags = pos_tag(clean_tokens)
print("\nPOS Tags:\n", pos_tags)

# NOUN PHRASE CHUNKING
grammar = "NP: {<DT>?<JJ>*<NN.*>+}"
cp = RegexpParser(grammar)
tree = cp.parse(pos_tags)
print("\nChunk Tree:\n", tree)

# DEPENDENCY PARSING
doc = nlp(text)
print("\nDependency Parsing:\n")
```

```
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)

# STEMMING
stemmer = PorterStemmer()
stemmed = [stemmer.stem(word) for word in clean_tokens]

print("\nStemming:\n", stemmed)

# LEMMATIZATION
lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(word) for word in clean_tokens]

print("\nLemmatization:\n", lemmatized)
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
Original Text:
 Peacock is beautiful bird
Sparrow is a small bird
Dog is a loyal animal
Cat drinks milk daily
Sun gives us light
Moon shines at night
Boy plays in garden
Girl reads a book
Water is very important
Trees give us oxygen
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No: 92301733021**

```
Tokens:
 ['Peacock', 'is', 'beautiful', 'bird', 'Sparrow', 'is', 'a', 'small', 'bird', 'Dog', 'is', 'a', 'loyal', 'animal', 'Cat', 'drinks', 'milk', 'daily', 'Sun', 'gives', 'us', 'light', 'Moon', 'shines', 'at', 'night', 'Boy',

Clean Tokens:
 ['Peacock', 'beautiful', 'bird', 'Sparrow', 'small', 'bird', 'Dog', 'loyal', 'animal', 'Cat', 'drinks', 'milk', 'daily', 'Sun', 'gives', 'us', 'light', 'Moon', 'shines', 'night', 'Boy', 'plays', 'garden', 'Girl', 'reads

POS Tags:
 [('Peacock', 'NNP'), ('beautiful', 'JJ'), ('bird', 'NN'), ('Sparrow', 'NNP'), ('small', 'JJ'), ('bird', 'NN'), ('Dog', 'NNP'), ('loyal', 'VBZ'), ('animal', 'JJ'), ('Cat', 'NNP'), ('drinks', 'NNS'), ('milk', 'VBD'), ('da

Chunk Tree:
 (S
  (NP Peacock/NNP)
  (NP beautiful/JJ bird/NN Sparrow/NNP)
  (NP small/JJ bird/NN Dog/NNP)
  loyal/VBZ
  (NP animal/JJ Cat/NNP drinks/NNS)
  milk/VBD
  (NP daily/JJ Sun/NNP)
  gives/VBZ
  us/PRP
  (NP light/JJ Moon/NNP shines/NNS night/NN Boy/NNP)
  plays/VBZ
  (NP garden/NN Girl/NNP)
  reads/VBZ
  (NP book/NN Water/NNP)
  (NP important/JJ Trees/NNP)
  give/VBP
  us/PRP
  (NP oxygen/NN))
```

```
Dependency Parsing:

Peacock -> nsubj -> is
is -> ROOT -> is
beautiful -> amod -> bird
bird -> compound -> Sparrow

 -> dep -> bird
Sparrow -> nsubj -> is
is -> ccomp -> is
a -> det -> Dog
small -> amod -> bird
bird -> compound -> Dog

 -> dep -> bird
Dog -> nsubj -> is
is -> ccomp -> is
a -> det -> animal
loyal -> amod -> animal
animal -> compound -> Cat

 -> dep -> animal
Cat -> nsubj -> drinks
drinks -> ccomp -> is
milk -> compound -> Sun
daily -> amod -> Sun

 -> dep -> daily
Sun -> nsubj -> gives
gives -> ROOT -> gives
us -> dative -> gives
light -> amod -> Moon

 -> dep -> light
Moon -> dobj -> gives
shines -> ccomp -> gives
at -> prep -> shines
night -> pobj -> at

 -> dep -> night
Boy -> nsubj -> plays
plays -> ROOT -> plays
in -> prep -> plays
garden -> compound -> Girl

 -> dep -> garden
Girl -> pobj -> in
reads -> advcl -> plays
a -> det -> Water
book -> compound -> Water
```

```
 -> dep -> important
Trees -> nsubj -> give
give -> ccomp -> is
us -> dative -> give
oxygen -> dobj -> give

Stemming:
 ['peacock', 'beauti', 'bird', 'sparrow', 'small', 'bird', 'dog', 'loyal', 'anim', 'cat', 'drink', 'milk', 'daili', 'sun', 'give', 'us', 'light', 'moon', 'shine', 'night', 'boy', 'play', 'garden', 'girl', 'read', 'book',

Lemmatization:
 ['Peacock', 'beautiful', 'bird', 'Sparrow', 'small', 'bird', 'Dog', 'loyal', 'animal', 'Cat', 'drink', 'milk', 'daily', 'Sun', 'give', 'u', 'light', 'Moon', 'shine', 'night', 'Boy', 'play', 'garden', 'Girl', 'read', 'bo
```

2. **Write your observation for stemming and lemmatization you obtained for the sentences.**
   - Stemming cuts the word and makes it short, but sometimes the word is not correct.
   - Lemmatization gives the correct base word with proper meaning.
   - So, stemming is simple and fast, but lemmatization is better and more correct.